Differential analysis of proteomics data using R

Karin Schork, MPC, Ruhr-Universität Bochum, karin.schork@rub.de
Robin Grugel, MPC, Ruhr-Universität Bochum, robin.grugel@rub.de

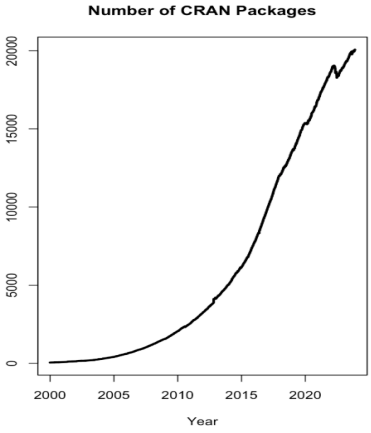
Kindly supported by
Julika Wenzel, MPC, Ruhr-Universität Bochum

Based on material by Michael Turewicz, Nils Hoffmann and Karin Schork

About R

What is R?

- **R** is a free software environment for statistical computing and graphics
- active development of **R** and open community
- over 26,000 additional packages available for all sorts of analysis



Where to get R

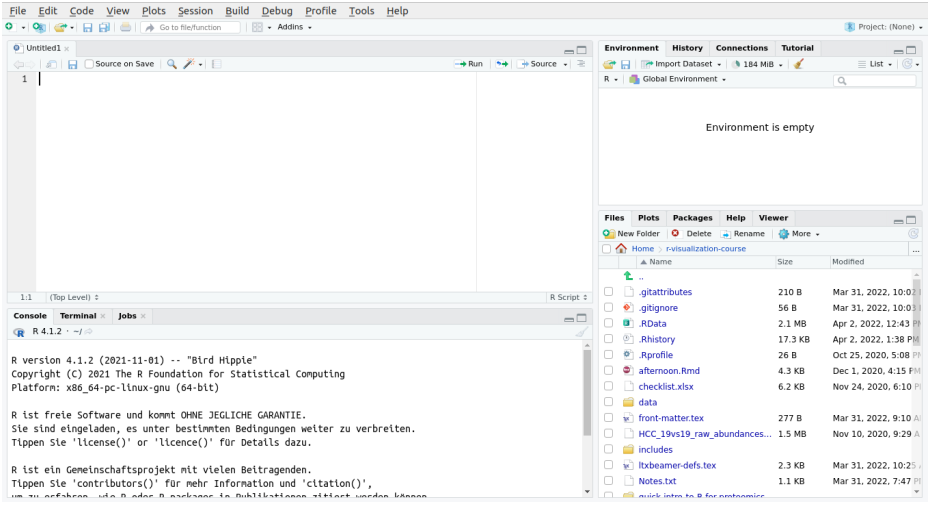
Comprehensive R archive network (CRAN)

- `http://cran.r-project.org`
- **R** for different operating systems
- **R** packages for diverse (statistical) methods
- documentation

BioConductor

- `https://www.bioconductor.org/`
- **R** packages for bioinformatics
- documentation + learning material
- contacts to community

Using R with RStudio



First steps in R

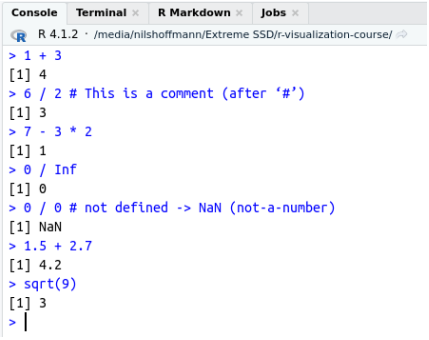


Figure 1: RStudio console as a pocket calculator

R as a pocket calculator

```
1 + 3
6 / 2 # This is a comment (after '#')
7 - 3 * 2
0 / Inf
0 / 0 # not defined -> NaN (not-a-number)
1.5 + 2.7
sqrt(9)
```

- multiplication and division are calculated before addition and subtraction ('Punkt-vor-Strich')
- plus ($+$) at beginning of line (instead of $>$): current command is not complete

- R console input is immediately evaluated when you hit 'Enter'

Assignments

Values and expressions can be assigned to named `objects`

- expressions combine values and operators / functions

`names of objects` in R

- may contain *letters*, *numbers*, period character '.' and underscore '_'
- have to start with a letter
- are case-sensitive

Save results in `objects`

```
# assignment, expression is evaluated
a.sum <- 3 + 5
# assignment of object value
a_number <- 4
# immediate printing of evaluated expr.
a.sum + a_number
```

```
## [1] 12
```

```
a.sum / 2
```

```
## [1] 4
```

```
# assignment of evaluated expression
difference <- a.sum - a_number
# printing of object value
difference
```

```
## [1] 4
```

Listing and removing objects

R saves objects in the so-called `workspace`

`ls()`	list all objects in the workspace
`rm("<name of object>")`	remove single object
`rm(list = ls())`	remove entire workspace

```
ls()
```

```
## [1] "a.sum"      "a_number"   "baseSize"   "difference"
```

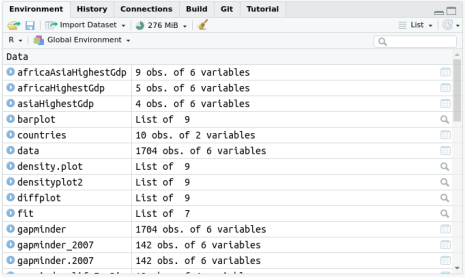


Figure 2: RStudio environment with objects

Closing your session (close R window or type `q()`):

Question to save current workspace (including all objects), if changed

```
q()
Save workspace image to /r-project/.RData? [y/n/c]:
```

Help

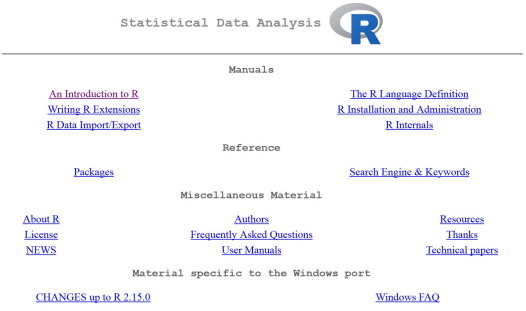
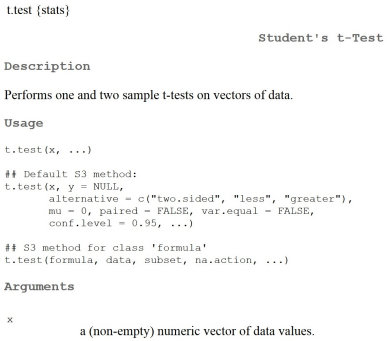


Figure 3: R Help Overview

```r
# open browser, start help system
help.start()
# help.search(<keyword>) or
# ??<keyword>
# show all help pages for <keyword>
help.search("t-test")
??"t-test"
# help(<function>) help on a function
# (exact name)
help(t.test)
# ?<function>, e.g.
?t.test
# browse package vignettes
browseVignettes(package="STRINGdb")
```

Help pages



t.test {stats} R Documentation

Student's t-Test

Description

Performs one and two sample t-tests on vectors of data.

Usage

```
t.test(x, ...)

## Default S3 method:
t.test(x, y = NULL,
       alternative = c("two.sided", "less", "greater"),
       mu = 0, paired = FALSE, var.equal = FALSE,
       conf.level = 0.95, ...)

## S3 method for class 'formula'
t.test(formula, data, subset, na.action, ...)
```

Arguments

x

a (non-empty) numeric vector of data values.

Figure 4: R Help Page

- Always have a look at help pages for functions you use
- Using R-help is a skill that requires practice

Sections of a help page (functions)

Description	short(est) description
Usage	e.g. order of arguments
Arguments	description of parameters
Details	detailed information
Value	output
See Also	similar functions
Examples	executable code

Basic Operators and Functions

`+, -, *, /, ^`	basic arithmetic operators
`max(), min()`	extreme values
`abs()`	absolute value
`sqrt()`	square root
`round(), floor(), ceiling()`	round (commercially, down, up)
`sin(), cos(), tan()`	trigonometrical functions
`log(), log10(), log2(), exp()`	logarithms
`sum(), prod()`	sum, product
`length()`	length of a vector (number of elements)
`Inf, -Inf`	infinity
`NaN`	not defined (Not a Number)
`NA`	missing values (Not Available)
`NULL`	'the null object in R', empty set

R - all vectors

- **R** is a vector-based language.
- A vector gathers data of a specific type in an ordered manner.
- `Data types` in **R**:

`logical`	`TRUE` or `FALSE`
`numeric [integer]`	integers
`numeric [double]`	real number (double precision)
`character`	letters and character strings

Vectors

The function `c()` (combine/concatenate) creates vectors:

```
x <- c(1.3, 7.4, 3.25)
x <- c(4.9, x, 3.95)

y <- c(TRUE, FALSE) # logical
z <- c("Alpha", "Bravo", "Charlie") # character
```

All arguments are coerced to a common type:

```
c(4, 3, y)
```

```
## [1] 4 3 1 0
```

```
c("Alpha", 3, TRUE)
```

```
## [1] "Alpha" "3"     "TRUE"
```

Coercion uses highest type in following hierarchy:

`logical < integer < double < complex < character`

Indexing vectors

- In order to navigate through vectors, matrices or data frames (indexing) use '[]':

```
x[3] # single entry
```

```
## [1] 7.4
```

```
x[1:3] # set of entries`
```

```
## [1] 4.9 1.3 7.4
```

```
x[c(1,3,12)]
```

```
## [1] 4.9 7.4  NA
```

```
x[-1] # exclude single entry via '-'
```

```
## [1] 1.30 7.40 3.25 3.95
```

```
x[-(1:3)] # exclude set of entries
```

```
## [1] 3.25 3.95
```

```
x[-c(1,3,12)] # exclude set of entries
```

```
## [1] 1.30 3.25 3.95
```

```
x[x<4] # entries meeting a logical condition
```

```
## [1] 1.30 3.25 3.95
```

```
index <- x>=4
x[index]
```

```
## [1] 4.9 7.4
```

Matrices - `data.frame`

- For data sets consisting of different types of variables use ‘data.frame()‘

```
Z <- data.frame("Gender" = c("f","f","m","f","m"),
                "Age"    = c(10,9,10,8,9),
                "Height" = c(120,118,104,109,111))
```

- Convention in R is: report subjects in rows, report variables, properties or traits in columns!
- Default row names are row numbers 1:n

	Gender	Age	Height
1	f	10	120
2	f	9	118
3	m	10	104
4	f	8	109
5	m	9	111

Indexing and handling of data frames and matrices

Y	1	2	...	C
1	1,1	1,2	...	1,C
2	2,1	2,2		
⋮	⋮			
R	R,1	R,2	...	R,C

General handling

`dimnames()`	get/set list of row and column names
`rownames(), colnames()`	get/set row/column names
`ncol(), nrow()`	number of columns/rows
`t()`	transpose (switch rows with columns)
`head(), tail()`	show first/last lines of a matrix

Indexing

```
Y[2,1] # 2nd row, 1st column
```

```
## [1] 2
```

```
Y[1,]  # complete 1st row
```

```
## [1] 3 5 6
```

```
Y[1:2,c(1,3)]
```

```
##      [,1] [,2]
## [1,]    3    6
## [2,]    2    1
```

```
# selection by column name
Z[,"Gender"]
```

```
## [1] "f" "f" "m" "f" "m"
```

```
Z$Gender
```

```
## [1] "f" "f" "m" "f" "m"
```

Logic and related functions

- Data type ‘logical‘ allows assessment of logical statements.

comparisons	
`==, !=, >, <, >=, <=`	
constants	
`TRUE, T, FALSE, F`	Don’t use T or F to name objects!
operators	
`!`	logical negation, NOT
`&&, ||`	AND, OR
`&, |`	AND, OR (for vectors)

```
(4 <= 1) && (6 == (3 * 2))
```

```
## [1] FALSE
```

```
c(TRUE, TRUE) & c(FALSE, TRUE)
```

```
## [1] FALSE  TRUE
```

```
FALSE || TRUE
```

```
## [1] TRUE
```

```
FALSE + TRUE + TRUE # TRUE = 1, FALSE = 0
```

```
## [1] 2
```

```
# functions
any(c(TRUE, FALSE)) => [1] TRUE
all(c(TRUE, FALSE)) => [1] FALSE
which(c(TRUE, FALSE)) => [1] 1
sum(c(TRUE, FALSE)) => [1] 1
```

Exercises

- We will use JupyterHub.NRW
- There is a link on the moodle

- Choose Ruhr-University Bochum
- Log-in with your RUB-ID and Password
- Accept 2x
- Choose RAM and CPU
- -> Start

